



Highly-fair bakery algorithm using symmetric tokens

Alex A. Aravind

Computer Science Program, University of Northern British Columbia, Canada

ARTICLE INFO

Article history:

Received 30 December 2009

Received in revised form 29 August 2010

Accepted 4 September 2010

Available online 18 September 2010

Communicated by S.E. Hambrusch

Keywords:

Concurrency

Distributed algorithm

Mutual exclusion

Bakery algorithm

Fairness

Bounded timestamps

ABSTRACT

This note proposes a new version of the bakery algorithm that: (i) assures fair tie-breaking when two or more processes choose same token number; and (ii) bounds the token numbers within the range $[-n, n]$. The algorithm is simple and uses one additional shared bit.

© 2010 Elsevier B.V. All rights reserved.

1. Introduction

Among the proposed solutions for the mutual exclusion problem, the bakery algorithm [8] is the most popular for its elegance, intuitive appeal, and power of satisfying many nice properties. The algorithm solves the problem using a very simple logic commonly used in practice. One downside of the bakery algorithm is that it requires unbounded size shared space to hold the token numbers and this downside is widely addressed in the literature [1,6,7,12, 14–16]. Another downside of the bakery algorithm is that, when two or more processes choose same token number due to concurrent computations, it breaks the tie using static priority. Using static priority in a mutual exclusion algorithm does not assure fairness, as it always favors a same set of processes, and hence apparently discouraged in the literature [4]. To the best of our knowledge, this fairness issue in the bakery algorithm has not been addressed so far.

Fairness is an important metric for many practical applications. When identical token numbers are obtained due to concurrent computations, the bakery algorithm always

favors processes with smaller ids. Concurrent token computations in the bakery algorithm may be less likely in uniprocessor systems. This may not be true in multiprocessor or multicore systems, which are becoming the norm nowadays, where processes can compute tokens in parallel. In that case, it is highly likely that two or more processes compute same token number, particularly during contention. In that case, although both processes have equal token numbers, one process is chosen over the other. This selection may not be noticeable in one or two concurrent competitions and may not even be considered as an unfairness issue. Between two processes i and j , if i is chosen over j whenever tie occurs, then it may be observed and considered unfair for the process j . Consider a typical scenario that, during their life time in the system, processes i and j ($i < j$) compete concurrently 1000 times. If i and j are executed in two different processors, the possibility of choosing identical token numbers is high in the bakery algorithm (i.e., i and j may concurrently read the token numbers of all other processes from the shared memory, compute the maximum plus 1 and write back to their token variables in the shared memory). Assume that during these 1000 concurrent competitions, 20 percentage of the time (i.e., 200 times) both i and j compute identical token numbers. In this case the bakery algorithm will

E-mail address: csalex@unbc.ca.

allow the process i to win over j in all the 200 tie-breaks, which we believe is not fair for the process j .

Mutual exclusion algorithms are typically used in operating systems, embedded systems, and most interactive real time applications that are expected to perform reliably for a longer period. In such cases, one cannot afford errors even with very small probabilities [2]. Although the problem of overflow is extremely unlikely with 64 bit arithmetic, the bakery algorithm might reach overflow on a 32 bit integer within a minute on a 3 GHz machine. Even if, in the future, most general purpose systems are expected to have 64 bit arithmetic, most embedded systems may well remain be at 32 bits for the sake of size and energy consumption. Recent studies indicate that the trend in embedded multicore processors and system is steadily increasing [10,13]. Therefore, we believe, simplicity and avoidance of overflow problem are important for a mutual exclusion algorithm to be widely adopted for such 32 bit systems in practice.

This paper presents a variant of the bakery algorithm, referred to as *symmetric token bakery algorithm*. The proposed algorithm uses positive and negative tokens, and these positive and negative tokens have the property of numerical symmetry. With symmetric tokens, we need only one additional shared bit. More importantly, the algorithm based on this numerical symmetry helps to: (i) break the tie fairly between any two processes; (ii) bound the token numbers within the interval $[-n, n]$; and (iii) simplify the logic of the algorithm.

A number of attempts have been made to bound the token numbers [1,6,7,12,14–16]. The modifications proposed in [7,16] are based on modulo arithmetic and they require the *less than* relation and the *maximum* function to be redefined. The explanation of these two redefined notions along with the use of modulo arithmetic is quite involved and hence the resulting algorithms are complex. Also, the modified algorithm proposed in [16] fails to assure the liveness property (defined in Section 2), and the algorithm proposed in [7] uses an additional integer shared variable. The modifications proposed in [1] make this algorithm non-symmetric and bound the token number by $3^n + 1$, which is quite large. Based on the proposal in [1], a modified algorithm was proposed in [15] to bound the tokens. Unfortunately, this algorithm cannot work if n is too big. The algorithms proposed in [12] use extra processes to bound the token values. The latest attempt to bound the token numbers was described in [14]. This version, referred to as the black-white bakery algorithm, is designed based on the ideas from the earlier works [11, 5,15]. This algorithm uses additional $n + 1$ binary shared variables and bounds the token numbers within the interval $[0, n]$.

2. System model and problem statement

The system has n asynchronous processes (or threads) communicating with each other only via shared memory using atomic read and write operations. We denote the ids of the processes as $1, 2, \dots, n$ and the shared resource which requires mutually exclusive access as R . The mutual exclusion problem is to design an algorithm that assures

the following properties: (i) At any time, at most one process is allowed to access R (*Safety property*); (ii) When some processes are interested in accessing R , one of them eventually succeeds in accessing R (*Liveness property*). In addition to these two properties, a desirable property is that: (iii) Any process interested in accessing R will be able to do so in finite time (*Freedom from starvation property*). Higher level fairness is most desirable for the algorithms to be used in practice. Some of the higher level fairness metrics are k -bounded overtaking (If a process p has requested access to R , then at most k other processes will access R before p does [2]), and doorway based first-in-first-out (FIFO) (If a process p completes the doorway (a segment of code in the entry section) before another process q does, then q cannot access R before p does [9]).

The following assumptions are made on the system: (i) The execution speed of any processor is finite but unpredictable; (ii) A process will neither access R continuously forever nor fail when it is accessing R ; and (iii) The solution must be symmetrical between n processes and as a result we are not allowed to introduce a static priority [4]. A solution to the mutual exclusion problem has essentially two components: *Entry* code and *Exit* code. That is, designing an entry code and its corresponding exit code constitutes the development of a mutual exclusion algorithm. The code segment of a process is divided into two parts: the code which accesses R , also referred to as critical section (CS), and the remaining code, noncritical section (NCS). For every process, say p , the entry and exit codes of the mutual exclusion algorithm have to be executed, respectively, before and after executing CS, as follows: *Process* $p ::= \text{while}(1)\{\text{NCS}(p); \text{Entry}(p); \text{CS}(p); \text{Exit}(p)\}$.

3. Symmetric token bakery algorithm

In the bakery algorithm, the tokens grow only in positive direction, possibly leading to unbounded growth. To mitigate the token growth, the symmetric token bakery algorithm allows token numbers to be chosen in both directions, positive and negative. An additional boolean bit d is used to indicate the direction of the token to be chosen at that moment. The direction alternates, similar to the black-white bakery algorithm, to control the token growth. In addition, the positive-negative symmetry is exploited to assure fairness in tie-breaking and to simplify the logic further.

The proposed algorithm uses the function $\max + 1$ (as in both bakery and black-white bakery algorithms) to compute a positive token number and, symmetrically, the function $\min - 1$ to compute a negative token number. The shared bit d , used to indicate the direction of the token value (positive or negative), has equal chance of taking value either 0 or 1, at any time. The relations $<$ and $>$, respectively, on the tokens are defined as follows: $(a, b) < (c, d)$ if $a < c$, or if $a = c$ and $b < d$; and $(a, b) > (c, d)$ if $a > c$, or if $a = c$ and $b > d$. There are four possible cases in determining the priority between two processes i and j in accessing R . (a) If the tokens $tk[i]$ and $tk[j]$ are positive and $(tk[i], i) < (tk[j], j)$, then i has higher priority over j – same as in the bakery algorithm. (b) If $tk[i]$ and $tk[j]$ are

Shared Variables: d : a binary bit. $c[n]$: a binary array. $tk[n]$: an array of values in $[-n, n]$. $\forall j, c[j] := 0; tk[j] := 0; d := 0.$ **Entry(i):**0. $c[i] := 1;$ // **Positive Direction**1. **if** ($d = 0$) {1.1. $tk[i] := \max\{tk[1], \dots, tk[n]\} + 1;$ 1.2. $c[i] := 0;$ 1.3. **for** $j := 1$ **to** n {1.4. **await** ($c[j] = 0$);1.5. **if** ($(tk[j] > 0) \wedge ((tk[j], j) \prec (tk[i], i))$) **await** ($tk[j] \leq 0$)1.6. **else await** ($tk[j] \geq 0 \vee (d = 1)$) }1.7. **if** ($d = 0$) $d := 1;$ }3. **CS**(i);**Exit(i):**4. $tk[i] := 0;$ **Negative Direction**2. **else** {2.1. $tk[i] := \min\{tk[1], \dots, tk[n]\} - 1;$ 2.2. $c[i] := 0;$ 2.3. **for** $j := 1$ **to** n {2.4. **await** ($c[j] = 0$);2.5. **if** ($(tk[j] < 0) \wedge ((tk[j], j) \succ (tk[i], i))$) **await** ($tk[j] \geq 0$)2.6. **else await** ($tk[j] \leq 0 \vee (d = 0)$) }2.7. **if** ($d = 1$) $d := 0;$ }**Fig. 1.** Symmetric token bakery algorithm.

negative and $(tk[i], i) \prec (tk[j], j)$, then j has higher priority over i – symmetric to the case (a). (c) If $d = 1$ and $tk[i]$ and $tk[j]$ are opposite token types, then the process with positive token has higher priority – analogous as in the black-white bakery algorithm. (d) If $d = 0$ and $tk[i]$ and $tk[j]$ are opposite token types, then the process with negative token has higher priority – symmetric to the case (c).

The system can be viewed as having two group of competing processes at any moment: a group that is allowed to access R next, referred to as the *active forum*, and the remaining processes, referred to as the *waiting forum*. The value of d indicates the current waiting forum. Every process interested in accessing R first enters the waiting forum of that time, computes its token number positive or negative depending on d value, and then proceeds further to determine its priority based on the tokens and forum status. All the processes in the active forum have higher priority than the processes in the waiting forum. When the current active forum becomes empty, the first process from the waiting forum, which has the highest priority at that time, referred to as the *leader* of that forum, flips d to swap forum status, and proceeds further to access R . Other processes in the active forum, referred to as *followers*, proceed one by one, following their leader, access R , and leave the competition. Some processes may quickly come back and choose a new token number, with opposite sign, and join the waiting forum for another access on R . Eventually, the current active forum will become empty, and the pattern repeats. The formal code of the symmetric token bakery algorithm, for process i , is given in Fig. 1.

Compared to the bakery algorithm, checking d at line 1 and checking and flipping d at line 1.7 are additional lines to facilitate the alternation. This is the main difference and

those lines are conceptually easy to follow. Computationally, the additional cost is 2 comparisons and 1 assignment on d . If these two lines are commented, then the line 1.6 and **else** part will become irrelevant to the algorithm and therefore can be ignored. Hence the remaining part of the code (lines 0, 1.1–1.5, 3, 4) is nearly identical to the bakery algorithm. When the comments are removed and the entire code is taken into account, the code segments 1.1–1.7 and 2.1–2.7 are symmetrical. That is, $\max + 1, >, <, \leq, \geq, d = 0$, and $d := 1$ in **if** part have been replaced, respectively, with their symmetric counterparts $\min - 1, <, >, \leq, \geq, d = 1$, and $d := 0$ in **else** part. At any time, a process will execute either 1.1–1.7 or 2.1–2.7, not both. This leaves us with the line 1.6 (correspondingly line 2.6), which is a straightforward conditional statement: when j is in the active forum, i waits for j to set $tk[j]$ either to 0 or to a number with the same sign as its own (i.e., j joins the forum of i for another access on R).

3.1. Correctness proofs

Designing a correct concurrent program such as the mutual exclusion algorithm using read and write primitives is not always easy, as it involves asynchronous executions of several processes interacting through shared variables. Even a minor change could significantly impact the correctness and the behavior of the algorithm. Therefore, we first verified the correctness of the symmetric token bakery algorithm, for $n \leq 4$, using the SPIN verification system [3].

SPIN verification system was developed at Bell Labs and has been widely used for specifying and verifying concurrent and distributed systems. SPIN systematically checks the correctness by simulating all possible computations in

the entire state space of the specified concurrent program. The biggest challenge in using SPIN is the problem of state space explosion as the number of processes increases. The verification of the symmetric token bakery algorithm for 5 processes ran out of memory in our system with 4 GB memory.

We write $j \in S$ to denote the process j is executing any line in the set S of instructions. We use $a..b$ to refer to the instructions between a and b . We say a process j is **competing** for CS if $j \in [0, 1, 1.1..1.7, 2.1..2.7, 3, 4]$. The set of instructions $[0, 1, 1.1, 1.2]$ (correspondingly, $[0, 1, 2.1, 2.2]$ for negative tokens) is referred to as the **doorway**.

Theorem 3.1. *Symmetric token bakery algorithm assures safety. That is, $i \in [1.7, 2.7, 3, 4] \wedge j \in [1.7, 2.7, 3, 4] \Rightarrow i = j$.*

Proof. Assume the contrary that $i \neq j$.

- *Processes i and j have same token type:* If one process notices the token number of the other as nonzero, at line 1.1/2.1, while computing maximum/minimum, then it must compute its token number with priority at least one lower than the token number of the other, and hence it must subsequently be blocked at line 1.5/2.5. If neither process notices the token number of the other as nonzero, at line 1.1/2.1, then each must subsequently wait at line 1.4/2.4 for the other to come out of the doorway before comparing tokens at line 1.5. In that case, since process ids are totally ordered, either $(tk[i], i) < (tk[j], j)$ or $(tk[j], j) < (tk[i], i)$. Now, the process with the lower priority token must be blocked at line 1.5/2.5.
- *Processes i and j have opposite token types:* Consider that one process notices the token number of the other as nonzero, at line 1.6/2.6. Without loss of generality, assume that $tk[i] > 0$ and i notices $tk[j] < 0$ at line 1.6. Then, either i must be blocked at line 1.6 or $d = 1$. Suppose $d = 1$. Let t be the latest time at which d was set to 1 (at line 1.7). This implies, process i must have computed $tk[i]$, at line 1.1, before t and j must have computed $tk[j]$, at line 2.1, after t . In this case, process j must notice $tk[i] > 0$ and $d = 1$ at line 2.6 and therefore must be blocked there. Similar argument holds if $tk[i] < 0$ and i notices $tk[j] > 0$ at line 2.6.

The assertion follows as all cases lead to contradiction. $\square$

Theorem 3.2. *Symmetric token bakery algorithm assures liveness. That is, when one or more processes interested in executing CS, one of them eventually succeeds in executing CS.*

Proof. Since process ids are totally ordered, tokens of same type are totally ordered. Among the processes with positive tokens, the process with the smallest token, say i , will not be blocked at line 1.5 by any other process with the same token type. Similarly, among the processes with negative token, the process with the largest token, say j , will not be blocked at line 2.5 by any other process with the same token type. If $d = 1$, then i will not be blocked by j , at line 1.7, and hence i will eventually access CS. If $d = 0$,

then j will not be blocked by i , at line 2.7, and hence j will eventually access CS. $\square$

Theorem 3.3. *Symmetric token bakery algorithm satisfies doorway based FIFO.*

Proof. We need to show that if a process i completes the execution of the doorway before another process j starts then i completes CS execution before j does.

- *Processes i and j have same token type:* Process j must have noticed i 's token number because j entered the doorway only after i has crossed it. This implies j must have computed the token number with priority at least 1 lower than that of i 's. In that case, j must be blocked (due to token checking at line 1.6/2.6) until i completes its access on CS.
- *Processes i and j have opposite token types:* In this case, the value of d must have changed between the computation of $tk[i]$ and $tk[j]$. Let t be the time at which the value has changed. If $tk[i] > 0$ and $tk[j] < 0$, then d must have been set to 1, at line 1.7, by a process with a positive token, at time t . By the safety property, this is possible only if no process had a negative token at time t . Since only the process with a negative token can change d at line 2.7, and a process can compute a negative token only after t , d value will not be changed before i completes its CS access. This implies, j must notice $tk[i] > 0$ and $d = 1$ at line 2.6 and therefore be blocked there until i completes its CS access. Similar argument holds true if $tk[i] < 0$ and $tk[j] > 0$. $\square$

Theorem 3.4. *Symmetric token bakery algorithm bounds the token numbers within the interval $[-n, n]$.*

Proof. Let t be the time of the most recent execution of the assignment statement $d := 0$. This assignment would be possible either by initialization or by a process with a negative token number. By the safety property, a process with a negative token number will set $d := 0$ only when no process with a positive token number is competing. That is, at time t , the token numbers of the processes in the system must be either 0 or negative. This implies that the first positive token number computed after t must be 1. Since n is the maximum number of processes in the system and each process can compute its token number as the current maximum plus 1, no process can compute a token number greater than n before d is set to 1 (by the process with the smallest positive token number). The process with the smallest positive token number will set d to 1 only when no process with a negative token number is competing. Now, the first negative token number computed must be -1 . Since n is the maximum number of processes in the system and each process can compute its token number as the current minimum minus 1, no process can compute a token number less than $-n$ before d is set to 0 (by the process with the largest negative token number). The pattern repeats keeping the token numbers within the range $[-n, n]$. $\square$

3.2. Fairness

When processes compute distinct token numbers, the shared resource is accessed in the order of the token numbers. When two processes compute identical token numbers (such a possibility is high in multicore and multi-processor systems), the algorithm breaks the tie in favor of one process over the other. Fairness is a very general concept and exact formal definition do not exist. However, providing equal opportunity to win tie-breaks for every process during their life time could be an acceptable definition of fairness in this context. This is also consistent with Assumption (iii), given in Section 2. In the bakery algorithm, between two processes, the process with smaller id has 100% chance and the process with larger id has 0% chance of winning tie-breaks. That is, the worst and the average cases in the bakery algorithm are same for any particular process and different for different processes. In the symmetric token bakery algorithm, the chance of winning tie-breaks is same for all processes. In particular, every process has equal probability of winning ties and that is formally proved next.

Theorem 3.5. *Symmetric token bakery algorithm assures fair tie-breaking between any two competing processes.*

Proof. Consider that processes i and j ($i < j$) choose same token number, say tk . We need to prove that i and j have equal probability of winning the tie. If tk is positive, then by line 1.5, i wins the tie. If tk is negative, then by line 2.5, j wins the tie. So it is enough to prove that tk has $1/2$ probability of being positive and $1/2$ probability of being negative. In symmetric token bakery algorithm, choosing either type depends on d value at that time. Since the code segment [1.1..1.7] is symmetrical and exclusive to [2.1..2.7], every process has $1/2$ probability of entering and executing either segment. Specifically, the assignment statements $d := 1$ at line 1.7 and $d := 0$ at line 2.7 have equal probability of being executed. This implies, d has equal probability of holding either value 0 or 1. That is, tk has equal probability to be either type.

The worst case will occur if ties in token numbers occur always with one type of tokens for all the processes. If all the ties occur always with positive token numbers in the symmetric token bakery algorithm, then they will be broken always in favor of smaller ids. This behavior is same as the behavior of the bakery algorithm. Similarly, if all the ties occur always with negative token numbers in the symmetric token bakery algorithm, then they will be broken always in favor of larger ids. This behavior is symmetrical and equivalent to the behavior of the bakery algorithm. Since the symmetric token bakery algorithm alternates non-deterministically between positive and negative tokens, this worst case scenario is least likely to happen in practice. In summary, in the average case, the symmetric token bakery algorithm assures high fairness (equal probability/opportunity to win tie-breaks). In the rare worst case situations, the symmetric token bakery algorithm assures the same level of fairness as the bakery algorithm. $\square$

3.3. Performance and comparison analysis

Symmetric token bakery algorithm is conceptually closer to both bakery and black-white bakery algorithms than the other bounded versions of the bakery algorithm proposed in [1,7,12,15,16]. Therefore, we compare the symmetric token bakery algorithm only with the bakery and the black-white bakery algorithms to expose the basic differences.

- **Space complexity:** Both symmetric token and black-white bakery algorithms use bounded token numbers, whereas the bakery algorithm requires unbounded size shared space for token numbers. Symmetric token bakery algorithm uses 1 extra binary variable than the bakery algorithm and bounds the token numbers within the interval $[-n, n]$, while the black-white bakery algorithm uses $n + 1$ extra binary variables and bounds the token numbers within the interval $[0, n]$.
- **Time complexity:** Symmetric token bakery algorithm uses 2 extra checks (on d at lines 1 and 1.7/2.7) than the bakery algorithm per CS entry and one assignment (on d at line 1.7/2.7) per session change. No computational difference between the bakery algorithm and the symmetric token bakery algorithm in computing and checking token numbers. In busy wait based mutual exclusion algorithms, a couple of checks per CS entry is an insignificant overhead. So, the computational performance of the symmetric bakery algorithm is almost same as that of the bakery algorithm. The extra n variables in the black-white bakery algorithm inherit associated shared memory accesses and comparison costs. More precisely, token computation in the black-white bakery algorithm uses extra shared memory accesses and comparisons to check the color of the tokens. Also, the await condition of token checking in the black-white bakery algorithm is more involved than in both bakery and symmetric token bakery algorithms. That is, the algorithm has to explicitly check the token colors during the token computation and this token checking incurs extra shared memory accesses and computations. This also has some effect on complicating the original logic. Another minor difference is that every process in the black-white bakery algorithm writes on the shared variable *color* (in the exit code), while in the symmetric token bakery algorithm only the leader of a forum flips d (before accessing CS).
- **Simplicity:** Simplicity is one of the attractive properties of the bakery algorithm. The main components of the bakery algorithm are token computation and token checking. Token computation in the bakery algorithm uses *max* function whereas the symmetric token algorithm uses *max* or *min*. The functions *max* and *min* are symmetrical and also identical in terms of computation and shared memory accesses. So the simplicity of the bakery algorithm is preserved almost intact in the symmetric token bakery algorithm.
- The bakery algorithm does not use any multi-writer shared variable. This property is lost in the later bounded token versions including the black-white and

the symmetric token bakery algorithms. Also, the bakery algorithm works even if a read overlaps with a write on same shared variable. In that case, only the write operation must be performed correctly and the overlapping read may return any arbitrary value. Both black-white and symmetric token bakery algorithms assume atomicity on shared variables.

4. Conclusion

A new version of the bakery algorithm called symmetric token bakery algorithm is presented in this paper. The algorithm is simple, elegant, assures high fairness, and bounds the token numbers within the interval $[-n, n]$, with an extra shared bit.

References

- [1] U. Abraham, Bakery Algorithms, in: Proc. of the Concurrency, Specification and Programming Workshop, 1993, pp. 7–40.
- [2] M. Ben-Ari, Principles of Concurrent and Distributed Programming, Addison-Wesley, 2006.
- [3] M. Ben-Ari, Principles of the Spin Model Checker, Springer, 2008.
- [4] E.W. Dijkstra, Solution of a problem in concurrent programming control, Communications of the ACM 8 (9) (1965) 569.
- [5] M.J. Fisher, N.A. Lynch, J.E. Burns, A. Borodin, Distributed FIFO allocation of identical resources using small shared space, ACM TOPLAS 11 (1) (1989) 90–114.
- [6] A. Israeli, M. Li, Bounded time-stamps, Distributed Computing 6 (4) (1993) 205–209.
- [7] P. Jayanti, K. Tan, G. Friedland, A. Katz, Bounding Lamport's Bakery Algorithm, in: Proceedings of the SOFSEM, in: LNCS, vol. 2234, 2001, pp. 261–270.
- [8] L. Lamport, A new solution of Dijkstra's concurrent programming problem, Communications of the ACM 17 (8) (1974) 453–455.
- [9] N. Lynch, Distributed Algorithms, Morgan Kaufmann Publishers, Inc., 1996.
- [10] M. Levy, T.M. Conte, Embedded multicore processors and systems, in: IEEE Micro, 2009, pp. 7–9.
- [11] J.M. Morris, A starvation-free solution to the mutual exclusion problem, Information Processing Letters 8 (2) (1979) 76–80.
- [12] M. Takamura, Y. Igarashi, Simple mutual exclusion algorithms based on bounded tickets on the asynchronous shared memory model, in: Proc. of the COCOON, in: LNCS, vol. 2387, 2002, pp. 259–268.
- [13] S. Tansley, Trends in embedded systems – a microsoft perspective, in: Proc. of the IEEE International Conference on Microelectronic Systems Education, 2007.
- [14] G. Taubenfeld, The black-white bakery algorithm and related bounded-space, adaptive, local-spinning and FIFO algorithms, in: Proc. of the DISC, in: LNCS, vol. 3274, 2004, pp. 56–70.
- [15] S. Vijayaraghavan, A variant of the bakery algorithm with bounded values as a solution to Abraham's concurrent programming problem, in: Proc. of Design, Analysis and Simulation of Distributed Systems, 2003.
- [16] T.-K. Woo, A note on Lamport's mutual exclusion algorithm, SIGOPS Operating Systems Review 24 (4) (1990) 78–81.